# Subset Sum Problem

*Given a set (or multiset) of integers, is there a subset whose sum is equal to a given sum.*

For example A = [3, 34, 4, 12, 5, 2] and sum = 26 then subsum(A, 26) = true as there is a subset {3, 4, 12, 5, 2} that sums up to 26.

### Approach 1: Naive

Naive algorithm would be to cycle through all subsets of N numbers and, for every one of them, check if the subset sums to the right number. The running time is of order `O(2^N . N)`, since there are `2` subsets and, to check each subset, we need to sum at most N elements.

### Approach 2: Recursive based on Knapsack 0/1

A better exponential time algorithm uses Recursion. Subset sum can also be thought of as a special case of the 0-1 Knapsack problem. For each item, there are two possibilities –

1. We include current item in the subset and recurse for remaining items with remaining sum.
2. We exclude current item from subset and recurse for remaining items.

Finally, we return true if we get subset by including or excluding current item else we return false. The base case of the recursion would be when no items are left or sum becomes negative. We return true when sum becomes 0 i.e. subset is found.

```
#include <bits/stdc++.h>
using namespace std;

class Solution{
    private:
        bool naiveUtil(vector<int> A, int N, int sum){
            if(sum==0)
                return true;
            if(N<0 || sum<0)
                return false;

            bool include = naiveUtil(A, N-1, sum-A[N]);
            bool exclude = naiveUtil(A, N-1, sum);

            return include || exclude;
        }
```

```
    public:
        bool naive(vector<int> A, int sum){
            return naiveUtil(A, A.size()-1, sum);
        }
};

int main(){
    Solution obj;
    int sum = 14;
    vector<int> A = {7, 3, 2, 5, 8};

    obj.naive(A, sum)==true?cout<<"Subset exists": cout<<"Subset does not
    exists";

    return 0;
}
```

The time complexity of above solution is exponential and auxiliary space used by the program is `O(1)`.

**Optimal substructure**

The problem has an optimal substructure. That means the problem can be broken down into smaller, simple "subproblems", which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

We can see that if we can find the whether a subset

`S={s1, s2,…,sm}` exists such that sum of elements equals to sum n

EITHER by excluding `sm` and using the solution of a smaller subset

`S = {s1. s2, …, sm-1}` that sums up to n

OR by including `sm` and using the solution of a smaller subset

`S = {s1. s2, …, sm-1}` that sums up to n-sm.

Let, `ss(S, n, m)` be the number of solutions of making n using first m numbers

`S={s1, s2,…,sm}.`

Then we can write the following recursive formula,

`ss(S, n, m) = ss(S, n, m-1) OR ss(S, n-sm, m-1).`

It is now clear that subset sum has a optimal substructure.

### Overlapping subproblems

```
set = { 7, 3, 2, 5, 8 }
sum = 14
```

Try to draw the recursion tree for the above set. Above solution also exhibits overlapping subproblems. If we draw the recursion tree of the solution, we can see that the same sub-problems are getting computed again and again.

Hence we can use Dynamic Programming to solve this problem.

### Top Down Dynamic Programming Approach

```
bool topDownUtil(vector<int> A, int N, int sum){
   //return true if sum becomes 0
   if(sum==0)
      return true;

   //return false if sum is -ve or we run out of elements
   if(N<0 || sum<0)
      return false;

   //create a key for the lookup table. Key = <A[i], Remaining sum/sum>
   //The lookup table contains the info whether we can make a subset for the
sum/remaining sum
   string key = to_string(N) + "|" + to_string(sum);

   if(lookup.find(key)==lookup.end()){
      bool include = topDownUtil(A, N-1, sum-A[N]);
      bool exclude = topDownUtil(A, N-1, sum);
      lookup[key] = (include || exclude);

      return include || exclude;
   }
   return lookup[key];
}
```

**Time Complexity:** O(n X sum)

**Space Complexity:** O(n X sum)

**Bottom Up Dynamic Programming Approach**

Video explanation: ideserve

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| 0 | T | F | F | F | F | F |
| 1 | T | T | F | F | F | F |
| 3 | T | T | F | T | T | F |
| 9 | T | T | F | T | T | F |
| 2 | T | T | T | T | T | T |

The first column is T becoz all the subsets of{0,1,3,9,2} have an empty set which sums upto 0.

The first row is F because an empty set cannot form a sum of 1,2,3,4,5

```
bool bottomUp(vector<int> A, int sum){
      //create a lookup table of size (N+1, sum+1)
      int N = A.size();
      bool lookup[N+1][sum+1];

      //fill up the table for target sum 0
      //all the elements can make a target sum 0.
      for(int i=0; i<=N; i++){
            lookup[i][0] = true;
      }

      //fill up the table for element 0.
      //0 can only make the target sum 0 but not all target sum.
      //Note: even if 0 is not there in our input set, we add it just to make
      //computations easy
      for(int i=1; i<=sum; i++){
            lookup[0][i] = false;
      }
```

```
    for(int i=1; i<=N; i++){
        for(int j=1; j<=sum; j++){
            if(j-A[i-1]<0)
                lookup[i][j] = lookup[i-1][j];
            else
                lookup[i][j] = lookup[i-1][j-A[i-1]] || lookup[i-1][j];
        }
    }
    return lookup[N][sum];
}
```

**Time Complexity:** O(n X sum)

**Space Complexity:** O(n X sum)